

What do we mean by “coding against an interface rather than a class?”

Coding against an **interface rather than a class** means that the code should depend on the **contract defined by an interface**, instead of depending on a specific class that provides the implementation.

An interface describes **what operations or behaviors are available**, without specifying the detailed implementation of those operations.

:This approach makes the code

- **Less dependent** on specific classes
- **More flexible**, because different classes can implement the same interface
- **Easier to maintain**, because implementations can be changed without changing the code that uses the interface
- **Easier to extend**, because new implementations can be introduced without modifying existing dependent code
- **Easier to test**, because different implementations can be substituted when needed

:In simple terms

Depend on what an object promises to do, not on the specific class that does it

What do we mean by “coding against abstraction rather than concreteness?”

Coding against **abstraction rather than concreteness** means that our code should depend on **general concepts, contracts, or common behavior** instead of depending directly on a specific implementation.

An **abstraction** represents the important functionality while hiding unnecessary implementation details.

Concreteness refers to a specific implementation or specific class

:Therefore, coding against abstraction means

- We focus on **what the object provides**
- We avoid depending on **how the object is implemented**
- We reduce the dependency between different parts of the system
- We make it easier to replace or modify implementations
- We make the system more flexible and maintainable

:The main idea is

High-level code should depend on abstractions, not on concrete
.implementations

?What is Abstraction as a Guideline .3

Abstraction as a guideline means using abstraction as a design principle when developing
.software

It tells us to **hide unnecessary implementation details** and expose only the functionality
.that is important to the user or to other parts of the program

The goal is to make each part of the system easier to understand and use without requiring
.knowledge of its internal implementation

:As a guideline, abstraction encourages us to

.1 **Hide implementation details** that are not necessary for the user of the object

.2 **Expose only the required functionality** through a clear interface or public
.members

.3 **Reduce dependencies** between different parts of the program

.4 **Depend on general behavior rather than specific implementations**

.5 **Separate what an object does from how it does it**

.6 **Make changes to implementation without unnecessarily affecting other parts of**
.the system

?How can we implement abstraction through what we have studied

:We can implement abstraction using several OOP concepts

Interfaces .1

Interfaces define a contract that specifies what operations a class must provide without
.defining the complete implementation

Abstract Classes .2

Abstract classes allow us to define common structure and behavior while leaving some
.functionality to be implemented by derived classes

Encapsulation .3

Encapsulation hides the internal state and implementation details of an object and controls
.access to them through appropriate members

Polymorphism .4

Polymorphism allows code to work with objects through a common abstraction while
.allowing different implementations to provide their own behavior

:Therefore

Abstraction is a guideline that encourages us to focus on essential behavior, hide unnecessary details, and make our code depend on abstractions rather than concrete implementations

?What is Operator Overloading .4

Operator overloading is a feature in object-oriented programming that allows a programmer to **define or modify the behavior of an existing operator when it is used with objects of a specific class**

Operators such as arithmetic, comparison, and equality operators normally have predefined behavior for built-in data types

Operator overloading allows these operators to have a **specific meaning for user-defined types**

:The important points are

- .It allows operators to work with objects of a class
- .It gives an operator behavior that is appropriate for that class
- .It improves the readability and natural expression of operations involving objects
- .It is a form of **compile-time polymorphism**
- The original operator is not replaced globally; its behavior is defined for the relevant user-defined type
- .The language places rules and restrictions on which operators can be overloaded